```
    34| p register.total
=>#0    <main> at code/debugger/debugger_test.rb:32
```

The first line shows what file is being executed and what lines are being displayed. You then see about 10 lines of code (fewer here because we hit the end of the file), with a pointer to the line currently about to be executed. The final line, <main> at, shows the current self object at the beginning of the line and again, the line you are at.

Below that is a prompt at which you can type commands. The full list is at https://github.com/ruby/debug.

Control Flow

Most of what you will want to do is move through the program. The debugger will helpfully give you comments as to what your commands do, so when I type s at the next prompt, I also see # step command. The most common commands are:

- step or s. The step command moves to the next place the code can stop, even if that's on a different line than what is called from this line. If you put a number after the command as in s 5, the command moves that number of steps before stopping. If I type s at this prompt, I end up at the beginning of the next line; if I type s again, I jump to the sale_price method because that's the next thing invoked.
- next or n. The next command steps over or moves to the next line. It executes the entire line of code then stops. Again, you can add a number argument if you want to go more than one line at a time. If I type n at this prompt, I wind up at the beginning of the next line. If I type n again, I wind up at the beginning of the following line; the sale_price method already complete.
- continue or c. The continue command moves forward until the script ends or you hit a breakpoint or a binding.debug line. If I type c at the prompt, the program runs until the end because there are no other stopping points.

The difference between step and next is worth mentioning. If you have a line of code such as this:

```
register.add_item("paper", 2, register.sale_price(799, 25))
```

typing s will take you to the sale_price method, and you'll be able to walk through that. Typing n will run the entire line of code and take you to the next one.

To exit the debugger, the command is q. CONTROL-D will also work. If you are remote debugging and want to end the remote process without exiting the debugger, the command is kill. Both of those commands prompt for a confirmation. If you want to skip that, you can use q! or kill!, though it seems like the same number of characters typed either way.

Breakpoints

A breakpoint is a location in the code where the debugger stops execution and gives you control at the debugger prompt to inspect or navigate through the code. As we saw, binding.break creates a breakpoint, but the debugger provides alternate ways to denote breakpoints without adding lines to the code.

You can set a breakpoint from the debugger's command prompt with break n or b n, where n is the line number of the currently displayed file. There are a lot of other options including break FILENAME:n, which lets you set a breakpoint on a different file, break CLASSNAME#METHOD